

numpy

Basics

```
In [1]: import numpy as np
```

Erzeuge ein Array `a` mit den Werten

```
2 3 2 4
3 4 9 3
4 0 1 4
```

```
In [2]: a = np.array([2, 3, 2, 4, 3, 4, 9, 3, 4, 0, 1, 4])
```

Bestimme die Dimensionen des Arrays

```
In [3]: a.shape
```

```
Out[3]: (12,)
```

Bestimme die `len` des Arrays

```
In [4]: len(a)
```

```
Out[4]: 12
```

Arrayerzeugung

Erzeuge ein Array der Zahlen von 1 bis 5

```
In [5]: np.array(range(1,6))
```

```
Out[5]: array([1, 2, 3, 4, 5])
```

Erzeuge ein Array der Länge 10, sodass `a[0] = 2`, `a[9] = 5` und die Elemente des Arrays eine arithmetische Folge bilden

```
In [6]: np.linspace(2,5,10)
```

```
Out[6]: array([2.          , 2.33333333, 2.66666667, 3.          , 3.33333333,
               3.66666667, 4.          , 4.33333333, 4.66666667, 5.          ])
```

Erzeuge ein `(3, 3)`-Array, dessen Einträge alle 1 sind

```
In [7]: np.ones((3,3))
```

```
Out[7]: array([[1., 1., 1.],
               [1., 1., 1.],
               [1., 1., 1.]])
```

Erzeuge eine (4, 4) -Einheitsmatrix

```
In [8]: np.eye(4)
```

```
Out[8]: array([[1., 0., 0., 0.],
               [0., 1., 0., 0.],
               [0., 0., 1., 0.],
               [0., 0., 0., 1.]])
```

Mit Hilfe der eben erzeugten Einheitsmatrix erzeuge eine (2, 8) -Matrix, wobei jeder 5.Eintrag 1 ist und die anderen 0

```
In [9]: a = np.zeros((2,8))
a[:,4] = 1
a
```

```
Out[9]: array([[0., 0., 0., 0., 1., 0., 0., 0.],
               [0., 0., 0., 0., 1., 0., 0., 0.]])
```

Visualisierung

Erzeuge ein Array `x` der Länge 100, sodass `x[0] = 0`, `x[99] = 2π` und die Elemente des Arrays eine arithmetische Folge bilden

```
In [10]: x = np.linspace(0, 2*np.pi, 100)
```

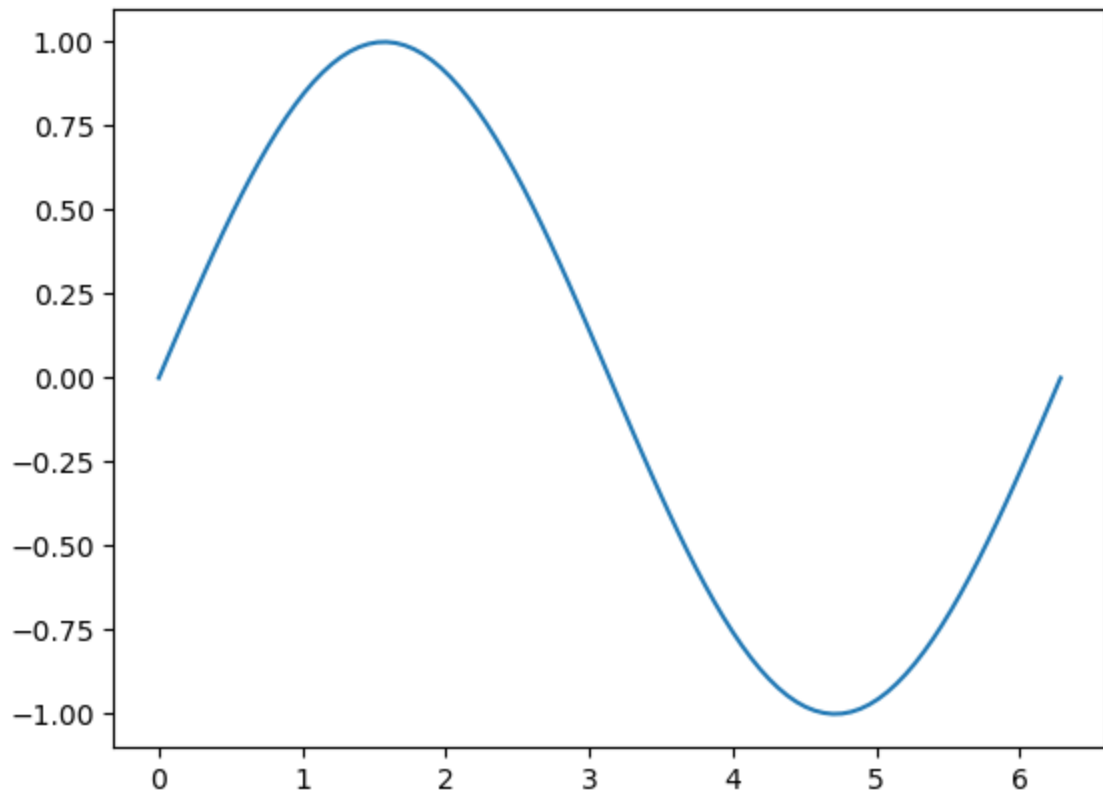
Erzeuge ein Array `y`, sodass `y[i] = sin(x[i])`

```
In [11]: y = np.sin(x)
```

Wir importieren *Seaborn* für einfache Graphiken und plotten unsere Funktion

```
In [12]: import seaborn as sns
sns.lineplot(x=x, y=y)
```

```
Out[12]: <Axes: >
```



Indizierung und Slicing

Erzeuge ein Array `a` mit den Werten

```
2 3 2 4
3 4 9 3
4 0 1 4
```

```
In [13]: a = np.array([[2,3,2,4],[3,4,9,3],[4,0,1,4]])
```

Selektiere die 2. Zeile

```
In [14]: a[1]
```

```
Out[14]: array([3, 4, 9, 3])
```

Selektiere die 3. Spalte

```
In [15]: a[:,2]
```

```
Out[15]: array([2, 9, 1])
```

Selektiere die Matrix

```
4 9 3
0 1 4
```

```
In [16]: a[1:3,1:]
```

```
Out[16]: array([[4, 9, 3],
               [0, 1, 4]])
```

Ermittle die Spannweite der Elemente von a

```
In [17]: np.ptp(a)
```

```
Out[17]: np.int64(9)
```

Erzeuge aus a den Output

True False True False

False False False False

False True True False

```
In [18]: result = np.zeros_like(a, dtype=bool)
result[0, [0,2,3]] = True
result[2, [1,2]] = True
result
```

```
Out[18]: array([[ True, False,  True,  True],
               [False, False, False, False],
               [False,  True,  True, False]])
```

Erzeuge aus a das Array

```
[2, 2, 4, 4, 4, 0, 4]
```

```
In [19]: a.flat[[0,2,3,5,8,9,11]]
```

```
Out[19]: array([2, 2, 4, 4, 4, 0, 4])
```

Erzeuge aus einer `zeros`-Matrix durch Slicing das Array

2 0 1 0

2 0 1 0

2 0 3 3

```
In [20]: a = np.zeros((3,4))
a[:,0] = 2
a[0:2,2] = 1
a[2,2:4] = 3
a
```

```
Out[20]: array([[2., 0., 1., 0.],
               [2., 0., 1., 0.],
               [2., 0., 3., 3.]])
```

Aggregatfunktionen

Erzeuge ein Array a mit den Werten

2 3 2 4

3 4 9 3

4 0 1 4

```
In [21]: a = np.array([[2,3,2,4],[3,4,9,3],[4,0,1,4]])
```

Berechne die Summe aller Werte

```
In [22]: np.sum(a)
```

```
Out[22]: np.int64(39)
```

Erzeuge ein Array, welches die Summe jeder Spalte enthält

```
In [23]: np.sum(a, axis=0)
```

```
Out[23]: array([ 9,  7, 12, 11])
```

Erzeuge ein Array, welches das Minimum jeder Zeile enthält

```
In [24]: np.min(a, axis=1)
```

```
Out[24]: array([2, 3, 0])
```